

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	4-Pthreads并行方程求解及蒙特卡洛	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.4.23

实验目的

- 掌握Pthreads 创建线程与合并线程的API。
- 掌握Pthreads 条件变量和互斥锁线程同步机制。
- 使用Pthreads实现一元二次方程求解和蒙特卡洛算法。
- 掌握并行求解的思想。

实验过程与核心代码

一元二次方程求解

```
typedef struct{
    double a,b,c;
    double x1,x2;
    double delta;
    double sqrt;
    pthread_cond_t cond;
    pthread_mutex_t mutex;
}equasion;
```

`equasion` 结构体，包括方程的三个系数，方程的两个解，前两个线程的计算结果，条件变量，互斥锁。

```

int main(int argc, char *argv[]){
    double a = atoi(argv[1]);
    double b = atoi(argv[2]);
    double c = atoi(argv[3]);

    equation eq = {a,b,c,0,0,-1,-1};
    pthread_cond_init(&eq.cond,NULL);
    pthread_mutex_init(&eq.mutex,NULL);

    pthread_t *thread1 = (pthread_t *)malloc(sizeof(pthread_t));
    pthread_t *thread2 = (pthread_t *)malloc(sizeof(pthread_t));
    pthread_t *thread3 = (pthread_t *)malloc(sizeof(pthread_t));

    clock_t start_time = clock();

    pthread_create(&thread1,NULL,compute_delta,&eq);
    pthread_create(&thread2,NULL,compute_sqrt,&eq);
    pthread_create(&thread3,NULL,compute_ans,&eq);

    pthread_join(thread1,NULL);
    pthread_join(thread2,NULL);
    pthread_join(thread3,NULL);
}

```

首先，从命令行输入中获取方程的三个系数，初始化结构体。创建三个线程，分别用于求解 `delta`，`delta` 的平方根，以及最终的解。使用上一个实验中的方法创建与合并线程。

在这个实验中，由于三个结果必须按顺序返回，因此需要使用条件变量与互斥锁实现同步。在这里先初始化条件变量与互斥锁。

```

void* compute_delta(void *arg){
    equation *eq = (equation*)arg;
    eq->delta = eq->b * eq->b - 4 * eq->a * eq->c;
    // signal to main thread
    pthread_cond_signal(&eq->cond);
    return NULL;
}

```

计算 `delta` 的函数。按照公式中的方法计算 `delta`，计算完成后发送信号，唤醒等待 `cond` 的线程。

```

void* compute_sqrt(void *arg){
    equation *eq = (equation*)arg;
    // add lock on eq,
    pthread_mutex_lock(&eq->mutex);
    while(eq->delta==-1){
    // if delta not computed, wait
        pthread_cond_wait(&eq->cond,&eq->mutex);
    }
    eq->sqrt = sqrt(eq->delta);
    // signal to main thread
    pthread_cond_signal(&eq->cond);
    // release lock on eq, let other threads visit eq
    pthread_mutex_unlock(&eq->mutex);
    return NULL;
}

```

计算 `delta` 平方根的函数。计算之前，先给 `eq` 加锁，确保只有 `sqrt` 能访问 `eq`。如果此时 `delta` 还没被计算出来，就让 `sqrt` 在条件变量上等待，并释放 `mutex`。否则，计算 `sqrt`，计算完成后发送信号，唤醒等待 `cond` 的线程，并释放互斥锁。

```

void *compute_ans(void *arg){
    equation *eq = (equation*)arg;
    //      add lock on eq,
    pthread_mutex_lock(&eq->mutex);
    while(eq->sqr== -1){
//          if sqrt not computed, wait
        pthread_cond_wait(&eq->cond,&eq->mutex);
    }
    eq->x1 = (-eq->b + eq->sqr) / (2 * eq->a);
    eq->x2 = (-eq->b - eq->sqr) / (2 * eq->a);
    //      release lock on eq, let other threads visit eq
    pthread_mutex_unlock(&eq->mutex);
    return NULL;
}

```

计算最终结果的函数。同样在计算前加锁。如果此时 `sqr` 还没被计算出来，就让 `ans` 在条件变量上等待，并释放 `mutex`。否则，计算 `ans`，计算完成释放互斥锁（是最后一个进程，不需要发送信号）。

```

clock_t end_time = clock();
double runtime = ((double)(end_time-start_time))/CLOCKS_PER_SEC;

if(eq.delta>=0){
    printf("answer: x1 = %.6f,x2 = %.6f, runtime = %.6fs\n",eq.x1,eq.x2,runtime);
}
else{
    printf("no real answer,runtime = %.6fs\n",runtime);
}

pthread_cond_destroy(&eq.cond);
pthread_mutex_destroy(&eq.mutex);
//free(thread1);
//free(thread2);
//free(thread3);

return 0;

```

结果输出部分。

蒙特卡洛算法

```
int main(int argc, char *argv[]){
    int n = atoi(argv[1]);
    int num_threads = atoi(argv[2]);

    int point_thread = n / num_threads;

    pthread_t *threads = (pthread_t *)malloc(num_threads *
sizeof(pthread_t));
    Data *data = (Data*)malloc(num_threads * sizeof(Data));

    clock_t start_time = clock();

    for(int i=0; i<num_threads; i++){
        data[i].points = point_thread;
        pthread_create(&threads[i], NULL, es_pai, &data[i]);
    }

    double point_in_all = 0;
    for(int j=0; j<num_threads; j++){
        pthread_join(threads[j], NULL);
        point_in_all += data[j].points_in;
    }

    double pai = 4 * point_in_all / n;
```

首先，从输入中获取点的总数和线程数。使用pthread API创建线程，合并线程。

```
typedef struct{
    int points;
    int points_in;
}Data;

void* es_pai(void *arg){
    srand(time(NULL));
    Data *data = (Data *)arg;
    int points_in = 0;

    for(int i=0; i<data->points; i++){
        double x = (double)rand() / RAND_MAX;
        double y = (double)rand() / RAND_MAX;
        if(x*x + y*y <= 1){
            points_in++;
        }
    }

    data->points_in = points_in;
}
```

在并行时，每个线程会生成一部分点，并记录这些点在单位圆中的个数。并行结束后，将各个线程中圆内的点加起来，得到最终结果。

实验结果

一元二次方程求解

```
paimon@PCLab1:~$ ./lab4-1 1 -6 9
answer: x1 = 3.000000,x2 = 3.000000, runtime = 0.000493s
```

有两个相同的实数解的情况。

```
paimon@PCLab1:~$ ./lab4-1 1 2 5
no real answer, runtime = 0.000344s
```

无实数解的情况。

```
paimon@PCLab1:~$ ./lab4-1 1 -5 6
answer: x1 = 3.000000,x2 = 2.000000, runtime = 0.000323s
```

有两个不同的实数解的情况。

蒙特卡洛算法

```
paimon@PCLab1:~$ ./lab4-2 10000 4
total points : 10000, points in circle: 7892.000000, estimated pai : 3.156800,
runtime: 0.001157paimon@PCLab1:~$
```

n=10000时估计的PAI值。

实验感想

在本次实验中，我学到了pthread的两个同步原语，`pthread_mutex`和`pthread_cond`

`pthread_mutex`：确保同一时刻只有一个线程可以访问共享资源，`pthread_cond`：允许一个线程在条件允许前等待，在条件允许后执行。

在实验中，我这次终于没有遇到segmentation fault的问题。在实现第一个算法时，我在使用同步原语时遇到了一些困难，通过复习操作系统知识解决了。另外，刚开始的时候，我把`sqrt`开始计算的条件设置为了`delta>0`，我输入的系数形成的方程又恰好是只有一个实数根的，这就导致我的程序无法继续运行。